DOKUMEN TEKNIS · PERPUSTAKAAN

Panduan Hosting Aplikasi Perpustakaan

Cara menaruh aplikasi Laravel “Perpustakaan” ke internet — dari shared hosting paling murah, VPS yang bisa diatur penuh, sampai layanan PaaS. Lengkap dengan langkah perintah, checklist keamanan, dan pemecahan masalah.

Stack · Laravel 13 · PHP 8.3 · SQLite/MySQL
Target · Shared hosting · VPS · PaaS
Versi · 1.0 — dibuat untuk produksi
Isi · 9 bab · perintah siap tempel

Daftar Isi

01 Sebelum Mulai — apa yang perlu disiapkan 03

02 Memilih Tempat Hosting 04

03 Opsi A — Shared Hosting (cPanel) 05

04 Opsi B — VPS (Ubuntu + Nginx + PHP-FPM) 07

05 Opsi C — Layanan PaaS (Laravel Cloud & sejenis) 11

06 Konfigurasi Database (SQLite & MySQL) 12

07 Email, Antrean, dan Penjadwal (Cron) 13

08 Checklist Keamanan Produksi 14

09 Pemecahan Masalah & Perawatan 15

1 · Sebelum Mulai

Aplikasi ini adalah proyek Laravel biasa. Untuk “naik ke internet” kamu cuma butuh tiga hal: **PHP**, **web server**, dan **database**. Panduan ini menjelaskan cara menyiapkan ketiganya di tiga jenis tempat hosting.

1.1 Yang harus ada di server

KOMPONEN	VERSI MINIMUM	CATATAN
PHP	8.2+ (disarankan 8.3)	Ikuti versi di `composer.json`
Ekstensi PHP	lihat daftar di bawah	Wajib hampir semuanya
Composer	2.x	Untuk memasang dependensi
Node.js + npm	18+	Hanya untuk membangun aset (CSS/JS). Bisa dilakukan di laptop lalu di-upload.
Web server	Nginx / Apache	Harus mengarah ke folder `public/`
Database	SQLite 3 / MySQL 8 / MariaDB 10.4+	SQLite paling mudah; MySQL untuk beban besar

Ekstensi PHP wajib

```
bcmath  ctype  curl  fileinfo  gd  json  libxml  mbstring
openssl  pdo_sqlite (atau pdo_mysql)  sqlite3 (atau mysql)
tokenizer  xml  xmlreader  xmlwriter  zip
```

1.2 Dua cara menaruh kode

- **Cara “siap-pakai”** — paket source yang kamu terima sudah berisi aset hasil build (`public/build/`), jadi kamu tidak wajib memasang Node.js di server. Cukup `composer install` lalu jalan.
- **Cara “dari nol”** — kalau kamu mengubah CSS/JS, jalankan `npm install && npm run build` untuk membangun ulang aset.

> RINGKASNYA
>
> Paling cepat: **upload source** → `composer install` → **atur** `.env` → `php artisan migrate --seed` → **arahkan web server ke** `public/`. Selesai.

2 · Memilih Tempat Hosting

Tidak ada jawaban tunggal “yang terbaik”. Pilih sesuai kebutuhan, tenaga, dan kantong.

JENIS	COCOK UNTUK	KELEBIHAN	KEKURANGAN
Shared hosting (cPanel)	Perpustakaan sekolah/kecil, uji coba, anggaran minim	Murah (Rp15–50rb/bln), tanpa urus server, panel klik-klik	Akses terbatas, tak bisa pasang Node.js, kontrol terbatas
VPS (Ubuntu + Nginx)	Produksi serius, data besar, butuh kendali penuh	Bebas atur apa saja, bisa pasang queue worker & cron, performa tinggi	Harus paham Linux; kamu yang rawat keamanan & update
PaaS (Laravel Cloud dll.)	Yang mau cepat & malas urus server	Deploy dari Git, auto-scaling, SSL otomatis, backup terkelola	Berbayar bulanan, sedikit “sihir” di balik layar

REKOMENDASI

Untuk **perpustakaan sungguhan** (banyak pengguna, ada email notifikasi harian), pakai **VPS kecil (1–2 GB RAM)**. Untuk **coba-coba atau perpustakaan kecil**, **shared hosting** sudah cukup — asal paketnya mendukung PHP 8.2+ dan SSH.

2.1 Perbandingan cepat biaya

OPSI	PERKIRAAN BIAYA	TENAGA SETUP
Shared hosting cPanel	± Rp20.000 – 60.000 / bulan	± 30 menit
VPS 1–2 GB	± Rp50.000 – 150.000 / bulan	± 2–3 jam (sekali saja)
Laravel Cloud / PaaS	± $5 – 20 / bulan	± 15 menit

Angka hanya gambaran umum; harga tiap penyedia berbeda.

3 · Opsi A — Shared Hosting (cPanel)

Paling murah dan paling mudah. Cocok kalau kamu hanya butuh aplikasi jalan tanpa ingin menyentuh server. Syarat: paket menyediakan **PHP 8.2+**, **MySQL**, dan sebaiknya **akses SSH**.

3.1 Ringkasan langkah

1. Unggah source ke server (lewat File Manager atau SSH).
2. Atur domain/subdomain agar document root-nya menunjuk ke folder `public`.
3. Pasang dependensi dengan Composer.
4. Buat database & isi berkas `.env`.
5. Jalankan migrasi + seeder, lalu optimasi.

3.2 Unggah berkas

Struktur di server yang disarankan (taruh kode di luar `public_html` demi keamanan):

```
/home/namamu/
├── perpustakaan/            # seluruh source (app, config, vendor, dll.)
│   ├── app/  config/  database/  public/  ...
└── public_html/             # document root domain
    └── (isi dari perpustakaan/public/)
```

Kalau penyedia hanya memberi satu folder, taruh semua di dalamnya dan atur document root subdomain ke `perpustakaan/public`. Lewat cPanel:

1. Buka **Domains → Create A New Domain** (atau Subdomains).
2. Isi nama domain, lalu ubah **Document Root** menjadi `/home/namamu/perpustakaan/public`.
3. Simpan. Domain sekarang menunjuk langsung ke folder publik aplikasi.

> PENTING
>
> **Jangan** mengarahkan document root ke folder induk (`perpustakaan/`). Kalau begitu, berkas `.env` , kode sumber, dan database bisa diunduh siapa pun. Document root **harus** `public/` .

3.3 Pasang dependensi (SSH)

Masuk lewat SSH, lalu:

```
cd ~/perpustakaan
composer install --no-dev --optimize-autoloader
cp .env.example .env
php artisan key:generate
```

Kalau **tidak ada SSH**: jalankan `composer install` di laptop, lalu unggah seluruh folder `vendor/` lewat File Manager (memakan waktu, tapi bisa).

3.4 Buat database & isi .env

1. cPanel → **MySQL Databases** → buat database + user, beri ALL PRIVILEGES.

2. Catat: nama database, user, password, host (biasanya `localhost`).
3. Edit `.env` di server (File Manager → klik kanan → Edit):

```
APP_NAME="Perpustakaan"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://perpustakaan.sekolahku.sch.id

DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=namamu_perpus
DB_USERNAME=namamu_perpus
DB_PASSWORD=passwordkuatmu

SESSION_DRIVER=database
QUEUE_CONNECTION=database
CACHE_STORE=database

MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=emailmu@gmail.com
MAIL_PASSWORD=app-password-gmail
MAIL_FROM_ADDRESS="perpustakaan@sekolahku.sch.id"
MAIL_FROM_NAME="${APP_NAME}"
```

> TENTANG EMAIL
>
> `MAIL_MAILER=log` hanya menulis email ke berkas log (tidak terkirim). Untuk benar-benar mengirim notifikasi keterlambatan, isi SMTP seperti contoh di atas (Gmail butuh App Password, bukan password akun biasa).

3.5 Migrasi, seeder, dan optimasi

```
php artisan migrate --force
php artisan db:seed --force          # data contoh (opsional di produksi)
php artisan storage:link             # agar cover buku bisa diakses

php artisan config:cache
php artisan route:cache
php artisan view:cache
```

3.6 Penjadwal (cron) di cPanel

Notifikasi keterlambatan berjalan otomatis pukul 07:00 — ini butuh cron. Di cPanel buka **Cron Jobs**, tambahkan satu tugas berjalan **tiap menit**:

```
* * * * * cd /home/namamu/perpustakaan && php artisan schedule:run >> /dev/null 2>&1
```

> KALAU SHARED HOSTING TAK PUNYA CRON
>
> Sebagian paket murah mematikan cron. Alternatif: panggil URL terjadwal dari layanan cron gratis (mis. cron-job.org) yang menembak endpoint pemicu, atau pindah ke VPS. Tanpa cron, fitur email keterlambatan otomatis tidak akan berjalan.

4 · Opsi B — VPS (Ubuntu + Nginx + PHP-FPM)

Cara paling fleksibel dan paling “benar” untuk produksi. Panduan ini memakai **Ubuntu 22.04/24.04**, **Nginx**, **PHP 8.3-FPM**, dan **MySQL**. Kalau mau lebih ringan, ganti database ke SQLite (lihat Bab 6).

4.1 Masuk & perbarui server

```
ssh root@IP_SERVER

apt update && apt upgrade -y
apt install -y nginx mysql-server unzip git curl
```

4.2 Pasang PHP 8.3 + ekstensi

```
apt install -y software-properties-common
add-apt-repository ppa:ondrej/php -y
apt update
apt install -y php8.3-fpm php8.3-cli php8.3-mysql php8.3-mbstring \
  php8.3-xml php8.3-curl php8.3-zip php8.3-gd php8.3-bcmath php8.3-sqlite3

php -v                       # pastikan 8.3.x
```

4.3 Pasang Composer

```
curl -sS https://getcomposer.org/installer | php
mv composer.phar /usr/local/bin/composer
composer --version
```

4.4 Buat database MySQL

```
mysql -u root -p
-- di dalam prompt MySQL:
CREATE DATABASE perpustakaan CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'perpus'@'localhost' IDENTIFIED BY 'passwordkuatmu';
GRANT ALL PRIVILEGES ON perpustakaan.* TO 'perpus'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

4.5 Unggah aplikasi

Letakkan kode di `/var/www/perpustakaan`. Bisa lewat `scp`, `git clone`, atau `rsync` dari laptop:

```
mkdir -p /var/www/perpustakaan
cd /var/www/perpustakaan
# contoh via scp dari laptop:
# scp -r ./perpustakaan-source/* root@IP_SERVER:/var/www/perpustakaan/

composer install --no-dev --optimize-autoloader
cp .env.example .env
php artisan key:generate
```

4.6 Atur .env untuk produksi

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://perpustakaan.sekolahku.sch.id

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=perpustakaan
DB_USERNAME=perpus
DB_PASSWORD=passwordkuatmu
```

Lalu siapkan aplikasi:

```
php artisan migrate --force
php artisan storage:link
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

4.7 Izin folder

```
chown -R www-data:www-data /var/www/perpustakaan
find /var/www/perpustakaan -type d -exec chmod 755 {} \;
find /var/www/perpustakaan -type f -exec chmod 644 {} \;
chmod -R 775 /var/www/perpustakaan/storage /var/www/perpustakaan/bootstrap/cache
```

4.8 Konfigurasi Nginx

Buat berkas `/etc/nginx/sites-available/perpustakaan`:

```
server {
    listen 80;
    server_name perpustakaan.sekolahku.sch.id;
    root /var/www/perpustakaan/public;

    index index.php;
    charset utf-8;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
    }

    location ~ /\.(?!well-known).* { deny all; }
    client_max_body_size 20M;
}
```

Aktifkan & muat ulang:

```
ln -s /etc/nginx/sites-available/perpustakaan /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx
```

4.9 HTTPS gratis (Let’s Encrypt)

```
apt install -y certbot python3-certbot-nginx
certbot --nginx -d perpustakaan.sekolahku.sch.id
# pilih "redirect" saat ditanya agar HTTP otomatis dialihkan ke HTTPS
systemctl status certbot.timer    # perpanjangan otomatis
```

4.10 Penjadwal (cron) di VPS

```
crontab -e -u www-data
```

Tambahkan baris ini (satu baris, tiap menit):

```
* * * * * cd /var/www/perpustakaan && php artisan schedule:run >> /dev/null 2>&1
```

4.11 Antrean (queue) — opsional tapi disarankan

Notifikasi email bisa dikirim lewat antrean agar pengguna tak menunggu. Buat layanan systemd agar worker otomatis hidup:

```
nano /etc/systemd/system/perpustakaan-worker.service
```

```
[Unit]
Description=Perpustakaan Queue Worker
After=network.target

[Service]
User=www-data
Restart=always
WorkingDirectory=/var/www/perpustakaan
ExecStart=/usr/bin/php /var/www/perpustakaan/artisan queue:work --sleep=3 --tries=3

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable --now perpustakaan-worker
systemctl status perpustakaan-worker
```

SETIAP KALI UBAH KODE

Jalankan ulang: `php artisan config:cache && php artisan route:cache && php artisan view:cache` lalu `systemctl reload php8.3-fpm` dan `systemctl restart perpustakaan-worker`.

5 · Opsi C — Layanan PaaS

Kalau kamu malas mengurus server, pakai layanan yang menangani semuanya. Cukup sambungkan repositori Git, dan setiap kali kamu `push` aplikasi otomatis di-deploy.

5.1 Laravel Cloud (paling mulus untuk Laravel)

1. Daftar di **cloud.laravel.com** dan hubungkan akun GitHub.
2. Buat aplikasi baru → pilih repositori kode Perpustakaan.
3. Tambahkan **Database** (MySQL/Postgres) dari menu Resources.
4. Isi Environment Variables: `APP_ENV=production`, `APP_DEBUG=false`, kredensial database (otomatis terisi bila pakai database bawaan), dan setelan SMTP.
5. **Deploy**. SSL, domain, dan penjadwal ditangani otomatis.
6. Aktifkan **Scheduler** (untuk notifikasi 07:00) dan **Queue** di dashboard.

5.2 Alternatif lain

LAYANAN	COCOK UNTUK	CATATAN
Laravel Forge + VPS	Mau otomatisasi tapi tetap punya VPS	Forge mengurus Nginx, SSL, cron, deploy otomatis
Railway / Render	Deploy cepat dari Git	Perlu sedikit penyesuaian (build command & start command)
Vercel / Netlify	—	Kurang cocok: keduanya untuk situs statis/JS, bukan PHP penuh

> KUNCI PAAS
>
> Yang penting: **environment variable** (sama seperti isi `.env`), **build command** (`composer install --no-dev && npm ci && npm run build`), **start command** (diarahkan ke `public/index.php`), dan **aktifkan scheduler + queue**.

6 · Konfigurasi Database

Aplikasi ini bisa memakai dua jenis database. Pilih sesuai kebutuhan.

6.1 SQLite — paling mudah

Cocok untuk perpustakaan kecil hingga menengah. Tidak perlu server database terpisah, hanya satu berkas. Aplikasi ini sudah memakai SQLite secara bawaan.

```
DB_CONNECTION=sqlite
# DB_DATABASE=/var/www/perpustakaan/database/database.sqlite
```

Pastikan berkasnya ada dan bisa ditulis:

```
touch /var/www/perpustakaan/database/database.sqlite
chown www-data:www-data /var/www/perpustakaan/database/database.sqlite
php artisan migrate --force
```

> KELEBIHAN SQLITE
>
> Backup = cukup salin satu berkas. Tidak ada service database yang bisa tumbang. Sangat andal untuk beban kecil. Batas praktisnya: penulisan bersamaan yang tinggi.

6.2 MySQL / MariaDB — untuk beban besar

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=perpustakaan
DB_USERNAME=perpus
DB_PASSWORD=passwordkuatmu
```

Lalu:

```
php artisan migrate --force
php artisan config:cache
```

6.3 Pindah dari SQLite ke MySQL

Karena semuanya memakai migration Laravel, kamu cukup mengubah `.env` lalu menjalankan migrasi ulang di database baru. Bila sudah ada data penting, ekspor dulu:

```
# ekspor data dari aplikasi (fitur bawaan) atau:
php artisan migrate:fresh --seed --force   # ⚠ menghapus data lama
```

> HATI-HATI
>
> `migrate:fresh` **menghapus seluruh data**. Jangan dijalankan di server yang sudah berisi data nyata.

7 · Email, Antrean, dan Penjadwal

7.1 Email notifikasi

Dua notifikasi dikirim aplikasi: **reservasi siap diambil** dan **keterlambatan pengembalian**. Keduanya butuh konfigurasi SMTP yang benar.

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=emailmu@gmail.com
MAIL_PASSWORD=app-password-16-karakter
MAIL_FROM_ADDRESS="perpustakaan@sekolahku.sch.id"
MAIL_FROM_NAME="${APP_NAME}"
```

Uji cepat pengiriman:

```
php artisan tinker
# lalu jalankan:
Mail::raw('Tes', fn($m) => $m->to('tujuan@email.com')->subject('Tes'));
```

7.2 Penjadwal (cron)

Notifikasi keterlambatan dijadwalkan **setiap hari pukul 07:00**. Butuh satu baris cron (lihat Bab 3.6 atau 4.10). Perintahnya:

```
* * * * * cd /path/ke/perpustakaan && php artisan schedule:run >> /dev/null 2>&1
```

7.3 Antrean (queue)

Untuk mengirim email di latar belakang, jalankan worker (lihat Bab 4.11). Ini membuat halaman tetap cepat walau SMTP lambat.

8 · Checklist Keamanan Produksi

Sebelum diumumkan ke publik, pastikan semua kotak ini tercentang.

#	HAL YANG DICEK	NILAI YANG BENAR
1	Mode debug dimatikan	`APP_DEBUG=false`
2	Environment produksi	`APP_ENV=production`
3	Kunci aplikasi unik	sudah `php artisan key:generate`
4	Document root	menunjuk ke `public/` saja
5	Berkas rahasia tak terekspos	`.env` & `database/` di luar web
6	HTTPS aktif	sertifikat valid (Let’s Encrypt)
7	Izin folder	`storage` & `bootstrap/cache` bisa ditulis
8	Kredensial database	password kuat, user terbatas
9	Email teruji	SMTP benar, tes kirim berhasil
10	Cron berjalan	`schedule:run` tiap menit
11	Backup rutin	database + folder `storage/app`
12	Akun admin bawaan	password seeder sudah diganti

> JANGAN LUPA
>
> Akun contoh dari seeder (`admin@perpustakaan.test` dan anggota demo) memakai password default. **Segera ganti atau hapus** sebelum aplikasi dipakai sungguhan.

8.1 Perintah “rapikan produksi”

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan optimize          # alternatif ringkas untuk ketiganya
```

Setelah mengubah `.env`, selalu bersihkan lalu cache ulang:

```
php artisan config:clear && php artisan config:cache
```

9 · Pemecahan Masalah & Perawatan

9.1 Masalah umum

GEJALA	PENYEBAB & SOLUSI
Halaman putih / “500 Server Error”	Biasanya izin folder atau `.env` salah. Cek `storage/logs/laravel.log`. Perbaiki izin: `chmod -R 775 storage bootstrap/cache`.
“Vite manifest not found”	Aset belum dibangun. Jalankan `npm install && npm run build`, atau unggah folder `public/build/`.
Tampilan berantakan (tanpa CSS)	`public/build/` tidak ikut terunggah, atau `APP_URL` salah.
“No application encryption key”	Belum menjalankan `php artisan key:generate`.
Cover buku tidak muncul	Jalankan `php artisan storage:link`, cek izin `storage/app/public`.
Email tidak terkirim	SMTP salah, atau `MAIL_MAILER=log` belum diganti. Cek log & tes kirim.
Notifikasi 07:00 tak jalan	Cron belum dipasang, atau `schedule:run` gagal. Cek `crontab -l`.
“Too many requests” saat login	Perlindungan bawaan setelah 5 percobaan gagal. Tunggu 1 menit.

9.2 Perawatan berkala

- **Backup** — salin `database/database.sqlite` (bila SQLite) dan folder `storage/app/public` (cover buku) secara berkala. Untuk MySQL, gunakan `mysqldump`.
- **Perbarui** — jalankan `composer update` (dengan hati-hati) dan pembaruan keamanan sistem: `apt update && apt upgrade`.
- **Pantau log** — periksa `storage/logs/laravel.log` sesekali.
- **Hapus cache lama** — `php artisan cache:clear` bila perlu.

9.3 Contoh perintah backup cepat

```
# backup database SQLite
cp /var/www/perpustakaan/database/database.sqlite \
   ~/backup-perpus-$(date +%F).sqlite

# backup cover buku
tar czf ~/backup-cover-$(date +%F).tar.gz \
   /var/www/perpustakaan/storage/app/public
```

SELESAI

Dengan mengikuti bab ini, aplikasi Perpustakaan siap dipakai di internet — aman, cepat, dan notifikasinya berjalan otomatis. Simpan dokumen ini sebagai rujukan saat merawat server.